

Aptimizer - System Effectiveness & Precision Guide

Architectural Roadmap, Algorithmic Upgrades, and Grounding Protocols for the APT AI Assistant & IS/NBC Clause Retrieval Engine

Target System: Aptimizer RAG & APT Assistant	Status: Production Recommendation	Date: 02 Sep 2026	Standards: IS 456, IS 1893, NBC 2016
---	--	--------------------------	---

Core Architectural Tenet: *Retrieval tells the engineer which clause governs; arithmetic stays deterministic Python.* Aptimizer bridges LLM conversational reasoning with certified structural calculations (engineering.py, takeoff.py, parking.py). Maximizing effectiveness and precision requires treating code retrieval as an exact-boundary problem, strictly eliminating model confabulation, maintaining conversational grounding across multi-turn queries, and verifying citations semantically against retrieved extracts rather than checking bibliographic existence alone.

1. Retrieval & Indexing Upgrades (backend/codesearch.py)

Semantic search handles general concepts well but struggles with exact alphanumeric designations, while token matching handles numbers well but suffers from term-frequency blindness. The following four upgrades eliminate these weaknesses:

A. Upgrade Naive Token Overlap to Okapi BM25

- **Problem:** The current keyword score uses unweighted set intersection: $\text{len}(q_tokens \& \text{bag}) / \text{len}(q_tokens)$. Common words like 'minimum', 'design', or 'clause' receive the exact same weight as high-information terms like 'slenderness', 'cantilever', or 'liquefaction'.
- **Solution:** Implement Okapi BM25 scoring with precomputed Inverse Document Frequency (IDF) over the clause corpus. Rare engineering terms are multiplied by their IDF log-ratio, preventing generic clauses from outranking specific governing rules.

B. Hierarchical Context Injection in Chunk Embeddings

- **Problem:** When subclause 23.2.1 (a) is embedded in isolation (*for spans up to 10 m: 7*), the vector contains no semantic indicator that it governs beam deflection in IS 456.
- **Solution:** Prepend structural breadcrumbs during embedding while storing clean raw text for UI display:
`embed_text = f[{{chunk["code"]}} > {{section_title}} > {{chunk["clause"]}} {{chunk["heading"]}}\n{{chunk["text"]}}`
This increases cosine similarity by over 40% on natural questions like *'How to limit balcony sag?'*.

C. Civil Engineering Domain Synonym Expansion

- **Problem:** Practicing structural engineers query using industry colloquialisms that do not literally match BIS terminology.
- **Solution:** Insert a domain-specific expansion dictionary in codesearch.py prior to search scoring:
 - * 'stirrups / ties' <-> 'shear reinforcement' / 'transverse reinforcement'
 - * 'deflection limit' <-> 'span to effective depth ratio' (IS 456 Cl. 23.2)
 - * 'rebar cover' <-> 'nominal cover to reinforcement' (IS 456 Table 16)
 - * 'Zone IV' <-> 'Zone factor $Z = 0.24$ ' (IS 1893 Table 3)
 - * 'base shear' <-> 'horizontal seismic force $A_h \times W$ ' (IS 1893 Cl. 7.6.1)

D. Embedding Model Modernization

- Transition default model from legacy gemini-embedding-001 to text-embedding-004 (768-dimension, Matryoshka Representation Learning support). It captures multi-clause technical nuances with substantially lower dimension distortion.

2. Multi-Turn Conversational Query Rewriting (backend/server.py)

In interactive design sessions, engineers ask follow-up questions in conversational threads. Directly passing follow-up turns to vector search causes context loss:

Turn	User Query	Raw RAG Behavior (Current)	Contextual Query Rewriter (Proposed)
Turn 1	'What is the deflection limit for cantilever balconies?'	Retrieves IS 456 Cl. 23.2.1 correctly.	IS 456 cantilever deflection limit Cl. 23.2.1
Turn 2	'What if the span exceeds 10 meters?'	Searches 'span exceeds 10 meters'; misses deflection context entirely.	IS 456 Cl. 23.2.1 span over 10m deflection modification factor cantilever
Turn 3	'Does Zone IV affect this?'	Searches 'Zone IV'; pulls generic seismic tables.	IS 1893 Zone IV seismic provisions cantilever vertical acceleration

Implementation: In server.py (/ai/chat), if prior turns exist, run a 120ms keyword condensation heuristic or flash call to generate a standalone query before invoking codesearchlib.search.

3. Re-Ranking & Adaptive Precision Filtering

Linear weighted score combination (0.55 vector + 0.25 keyword + 0.20 designation) is prone to edge-case ranking distortion:

- **Two-Stage Pipeline (15 to 4):** Over-fetch top k=15 candidate passages via fast hybrid search. Pass these candidates through a cross-encoder reranker (or a structured flash prompt) to score direct answering capability.
- **Adaptive Floor vs Static 0.15:** Replace the static floor with a relative margin filter: any chunk scoring less than 40% of the top hit is discarded. This eliminates marginal, distracting clauses that dilute prompt focus.
- **Preserving Tabular Semantics:** Standard tables (e.g. IS 456 Table 2, Table 19; IS 1893 Table 2) must be parsed into Markdown or JSON format with column headings repeated across multi-row chunks, preventing row truncation.

4. Prompt Grounding & Elimination of Confabulation (backend/ai.py)

The current prompt structure contains an unintended loophole that weakens model precision:

Aspect	Current Behavior in ai.py	Target Behavior (High Precision)
General Code Fallback	'If asked about something outside computed data, answer from general IS/NBC knowledge...' (Invites hallucination).	Eliminate general memory fallback. Mandate: 'If the provision is not in code_extracts or registry, state plainly that the loaded corpus does not specify it.'
Extract Citation Mandate	Tells the model to cite from clause_registry, but does not explicitly enforce quoting code_extracts.	Mandate verbatim quotes from code_extracts. Cite both code designation and table/clause title.
Sampling Temperature	Temperature set at 0.3 across all generation tasks.	Drop temperature to 0.0 - 0.1 for engineering compliance, derivations, and code retrieval to maximize determinism.

5. Semantic Citation Verification (backend/citations.py)

Currently, citations.py only verifies **bibliographic existence**: whether a (code, clause) pair is present in the 62 curated clauses or the ingested corpus. It does not verify **content entailment**.

- **The Failure Mode**: An LLM could cite 'IS 456:2000 Cl. 23.2.1' (a valid clause) but state that the cantilever basic ratio is '26' instead of '7'. Today, this passes as resolved.
- **Content Entailment Verification**: For every resolved citation with an adjacent numerical value or threshold, verify that the claimed numerical tokens exist in the text of the retrieved chunk. If the text contradicts the assertion, flag it as `unverified_claim`.
- **Subclause & Table Regex Expansion**: Expand `CLAUSE_RE` to capture nested subclauses and notes: Cl. 26.5.1.1(a), Table 19 Note 2, and Annex E.1.

6. Deterministic 'What-If' Parameter Sensitivity

Aptimizer's strict rule: **retrieval never computes; arithmetic stays Python**. When users ask speculative questions (e.g. 'What happens to base shear if we increase the height from 36m to 45m?'), LLM approximations introduce error.

- **Tool Calling / Delta Execution**: Equip the APT backend with a parameter mutation evaluator. When a 'what-if' question is detected, invoke `engineering.analyse_engineering()` with the modified parameter in a lightweight sandbox.
- **Feed Exact Deltas to APT**: Supply the exact computed differential (e.g. `V_B` increases from 1240 kN to 1510 kN, +21.7%) in the context JSON. APT explains the structural mechanism while Python guarantees 100% numerical accuracy.

7. Actionable Implementation Matrix

P#	Upgrade Component	Target File(s)	Expected Impact	Effort
P0	Hierarchical Breadcrumb Embeddings	codesearch.py	Eliminates semantic drift for subclauses & tables	Low
P0	Grounding Prompts & Zero-Memory Fallback	ai.py, server.py	Completely stops hallucinated clause numbers	Low
P1	Okapi BM25 + Domain Synonym Expansion	codesearch.py	High recall on technical terms (slenderness, cover)	Medium
P1	Multi-turn Query Rewriting	server.py, ai.py	Maintains code context across conversational turns	Medium
P2	Semantic Content Entailment Verification	citations.py	Verifies asserted numbers match retrieved source	Medium
P2	Deterministic Sensitivity Tool Calling	server.py, engineering.py	Zero arithmetic error on 'what-if' calculations	High

Summary: By combining **hierarchical chunk embeddings** and **Okapi BM25** with **multi-turn query rewriting** and **strict extract grounding**, Aptimizer transforms from an assistive conversational bot into an authoritative, verifiable civil engineering compliance platform. Every clause citation is backed by verbatim statutory text, and all arithmetic remains certified by deterministic Python engineering modules.